PERSONAL INFORMATION MANAGEMENT (PIM) SYSTEM

Design Document

Document	Design Document
Version	1.4
Date	8 November 2026
Group	Group [number]
Status	Final

Group members

Name	Student ID
[Member 1 name]	[Student ID]
[Member 2 name]	[Student ID]
[Member 3 name]	[Student ID]
[Member 4 name]	[Student ID]

Contents

1 Introduction **3**

2 Architecture **3**
2.1 Chosen pattern: Model–View–Controller 3
2.2 How the components are instantiated 4

3 Structure of and relationships among the main code components **7**
3.1 Model classes 7
3.2 Controller and View classes 16

4 Example use: search for records, then update one **23**

5 Design decisions **24**
5.1 Platform and libraries 24
5.2 Structure 25
5.3 Data and rules 25
5.4 Interaction 26
5.5 Testing 27

6 Design quality **27**

List of Figures

1 Architecture of the PIM system. Solid arrows are calls; dotted arrows from `pim.py` show object creation; the dotted arrow from the View is its use of model value types. 4
2 Model classes, part 1: the Working Collection, the PIR classes, and alarms. Argument types are shortened; the tables give the full types. 7
3 Model classes, part 2: search criteria, built with the Composite pattern. 8
4 Model classes, part 3: the exception hierarchy. 8
5 Controller and View classes. Argument types are shortened; the tables give the full types. . 16
6 Sequence diagram: searching for Events whose description contains “COMP”, then changing the start of the match. 23

1 Introduction

This document describes the design of the command-line PIM system. Requirement ids such as FR-16 refer to the Software Requirements Specification (SRS), version 1.4.

- **Section 2** presents the architecture: the pattern chosen, the reasons for it, and how it is instantiated.
- **Section 3** describes the main code components: each class with its fields and public methods, and how the classes relate.
- **Section 4** follows one example use, *search for records and then update one*, with a sequence diagram.
- **Section 5** records the design decisions, each with its reason and the alternatives rejected.
- **Section 6** assesses the design for modularity, efficiency, extendibility, and justifiability.

The system is written in Python 3.12 and uses only the standard library. All type names are Python types: `X | None` means "an X, or nothing", `list[X]` is a list of X, and `dict[K, V]` is a mapping from K to V.

2 Architecture

2.1 Chosen pattern: Model–View–Controller

The system uses the **Model–View–Controller (MVC)** pattern. Each of the three components is a top-level Python package: `model`, `view`, and `controller`. Figure 1 shows the components and how they call each other.

Why MVC. There are four reasons.

1. **The brief asks for it.** All model code must be in a separate package named `model` so that it can be unit-tested. MVC draws exactly that line: the model holds the data and the rules and knows nothing about the screen.
2. **It fits an interactive program.** The PIM runs a loop: read one user action, change the data, show the new state. MVC splits that loop into three jobs. The View reads input and draws the screen, the Controller turns one finished action into model calls, and the Model applies the rules.
3. **Two front ends share one controller.** On an interactive terminal the View is a full-screen `curses` interface; for scripted input, such as the tests, it is a line-by-line interface. Both call the same `App`, so their behaviour cannot drift apart.
4. **Tests need no screen.** Search (US7), persistence (US10, US11), and alarms are tested by calling `PIM` directly. The current time `now` is passed in, so the alarm tests do not depend on the wall clock.

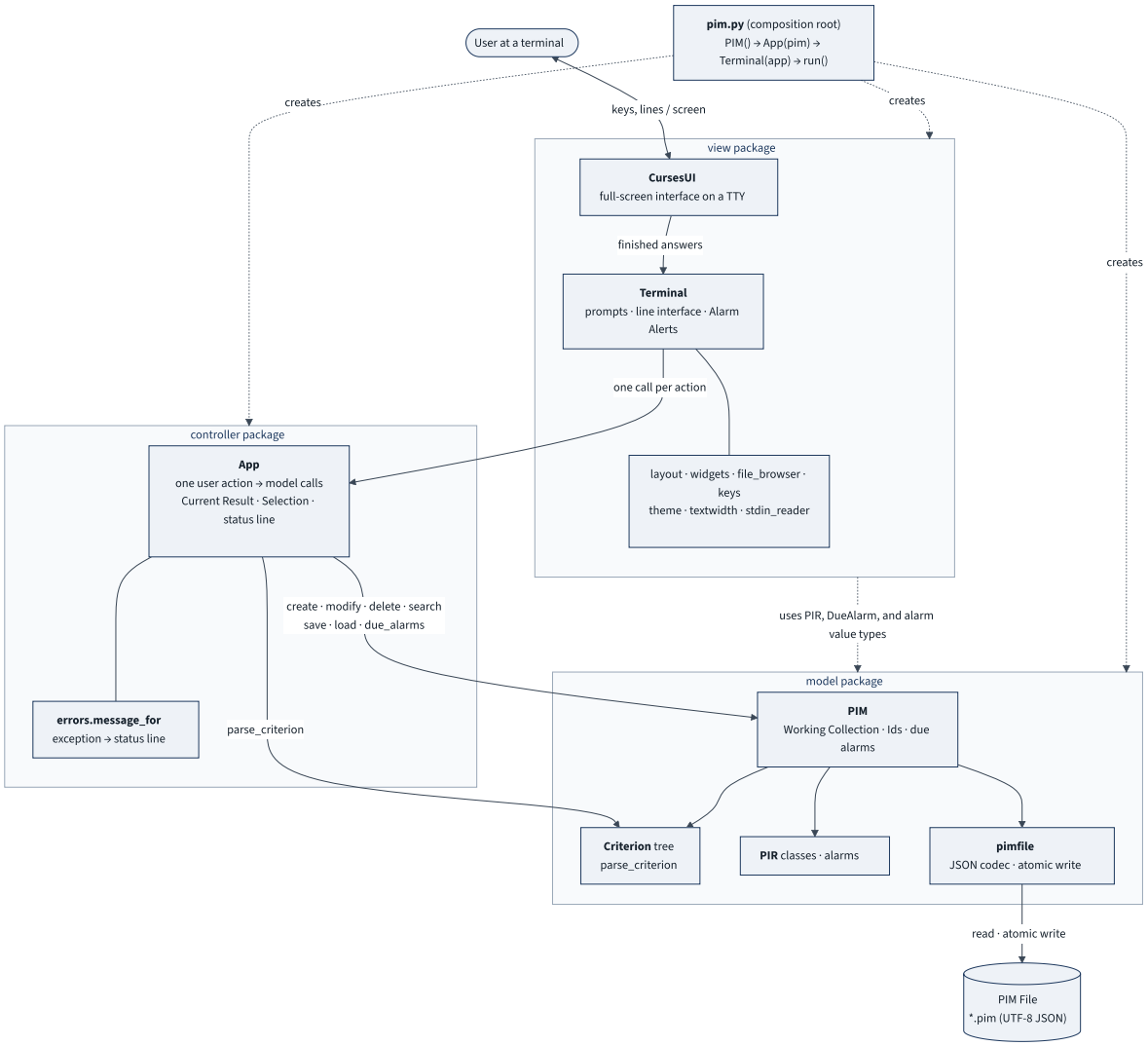


Figure 1. Architecture of the PIM system. Solid arrows are calls; dotted arrows from `pim.py` show object creation; the dotted arrow from the View is its use of model value types.

Alternatives considered

- *A layered architecture* (user interface, service, storage) would need a separate storage layer behind an interface. With one file format, that layer would have a single implementation and add indirection for no benefit. In this design, persistence is two methods of the model (`save` and `load`).
- *A single procedural script* would mix parsing, input, and screen code, and it would not meet the brief's requirement for a `model` package.

2.2 How the components are instantiated

Component	Package and classes	Responsibility in this system
Model	`model`: `PIM`, the PIR classes, the alarm classes, the criterion classes, `pimfile`	Holds the Working Collection and assigns Ids. Validates every field. Parses and evaluates Search Criteria. Reads and writes the PIM File. Computes the due alarms for a given `now`. Raises a subclass of `PIMError` for every rule violation.
Controller	`controller`: `App`, `errors`	Handles one completed user action: create, modify, delete, search, print, save, or load. Keeps the Current Result, the active criterion, and the Selection. Turns every model exception into one English status line, so no exception reaches the user as a traceback.
View	`view`: `Terminal`, `CursesUI`, and helper modules	Owns the event loop with its 500 ms tick. Reads keys or lines, asks for fields one at a time, and draws the title, the Alarm Alerts, the list, the details, and the status line. Remembers which alerts were dismissed.
Composition root	`pim.py`	`main()` creates `PIM()`, passes it to `App(pim)`, passes that to `Terminal(app)`, and calls `run()`.

Package layout

```
pim.py              composition root: python3 pim.py
model/              the unit-test surface; no threads, input, or screen code
  pim.py            PIM: Working Collection, Ids, dirty flag, save/load, due alarms
  pir.py            PIR classes, alarms, field specs, date-time parsing, exceptions
  criterion.py      search criteria (Composite pattern), tokenizer, parse_criterion
  pimfile.py        PIM File JSON codec and atomic write
controller/
  app.py            App: one user action -> PIM calls; Current Result, Selection
  errors.py         message_for: exception -> one status line
view/
  terminal.py       Terminal: prompts, wizards, Alarm Alerts, line interface loop
  curses_ui.py      CursesUI: full-screen loop on a TTY
  layout.py         screen model shared by both front ends
  widgets.py        Chooser, DateTimePicker, Prompt
  file_browser.py   FileBrowser for load and save-as paths
  keys.py, theme.py, textwidth.py, stdin_reader.py     smaller helpers
tests/unit, tests/integration, tests/e2e
```

Dependency rules

- `model` imports nothing from `view` or `controller`, and nothing outside the standard library.
- `controller` imports only from `model`.
- `view` sends every user action to `controller.App`. From `controller` it also uses the status constants, the help text, and `errors.message_for`.
- `view` uses `model` only for value types and helpers: it displays `PIR` and `DueAlarm` objects, builds `RelativeAlarm` and `AbsoluteAlarm` values from the alarm prompts, looks up a PIR class and its `FIELDS` to generate prompts, formats times with `format_datetime`, shows the clock in `HKT`, and catches `PIMError`.

- `view` holds no business rules and does not know the file format; `controller` never reads or writes files itself.

The interface of `model`. Other packages use only these names: `PIM`; `parse_criterion`; the classes `Note`, `Task`, `Event`, `Contact`, `RelativeAlarm`, `AbsoluteAlarm`, and `DueAlarm`; the exception classes; and the helpers `pir_class`, `format_datetime`, `HKT`, the field-kind constants, and `append_pim_extension` (which `App.save_target` uses to show the final file name before an overwrite question). Validation, matching, the tokenizer, and the JSON codec stay inside the package.

Event loop. The View runs the loop, and only its main thread calls the model.

- On a TTY, `CursesUI.loop` waits for a key with a 500 ms timeout.
- Otherwise, a daemon thread reads standard input into a `Queue`, and `Terminal` waits on the queue with a 500 ms timeout.

After each input or timeout, the View asks `App.due_alarms(now)` for alerts. The line interface redraws after every input line; on a timeout, either front end redraws only if its screen snapshot has changed. The snapshot covers the visible alerts, the Bound File and dirty flag, the Ids in the Current Result, the Selection, the printed text, the status line, and the open prompt; the full-screen snapshot adds the edit buffer, the widget state, the terminal size, and the clock minute. The reader thread only queues lines and never touches the model, so no lock is needed. Dismissed alerts are a set of (Event Id, alarm index) pairs in the View and are never written to the PIM File.

3 Structure of and relationships among the main code components

3.1 Model classes

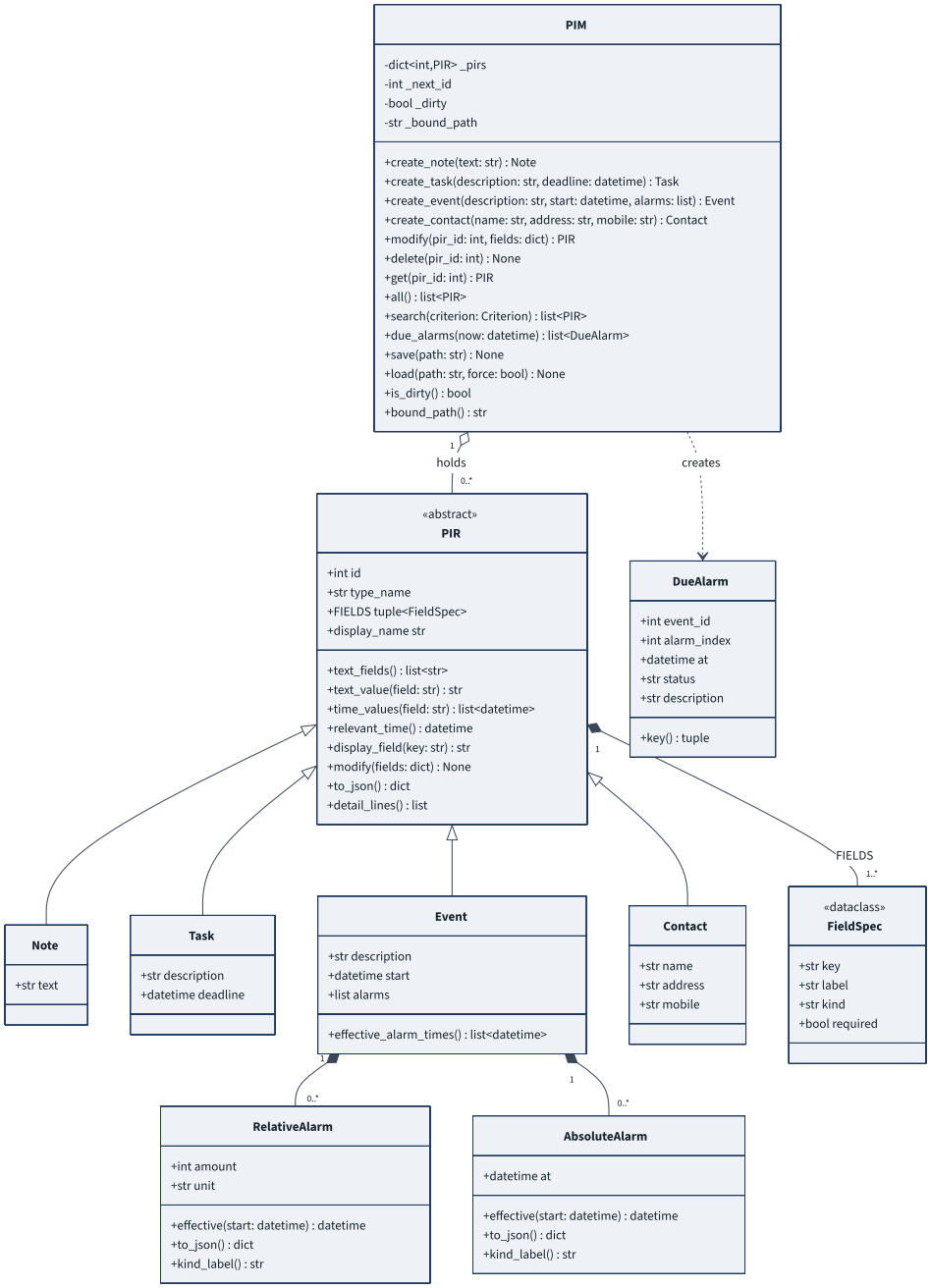


Figure 2. Model classes, part 1: the Working Collection, the PIR classes, and alarms. Argument types are shortened; the tables give the full types.

Relationships

- `PIM` *aggregates* any number of PIRs, keyed by Id.
- `PIR` is an abstract base class with four concrete subclasses. Its methods that a subclass must provide raise `NotImplementedError`. Each subclass declares its fields as `FieldSpec` values in `FIELDS`, from which the View builds the create and modify prompts.
- An `Event` is *composed of* zero or more alarms. Each alarm is a `RelativeAlarm` or an `AbsoluteAlarm`, and both provide `effective(start)`.
- `PIM.due_alarms` *creates* `DueAlarm` value objects.

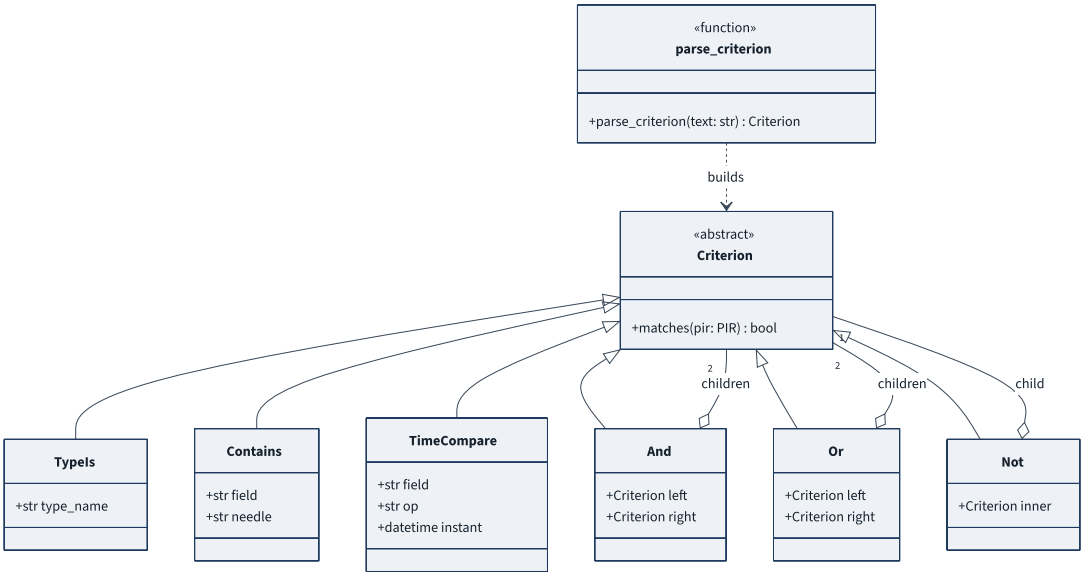


Figure 3. Model classes, part 2: search criteria, built with the Composite pattern.

Relationships

- A Search Criterion is a tree built with the **Composite** pattern. The leaves `TypeIs`, `Contains`, and `TimeCompare` test one PIR; `And`, `Or`, and `Not` combine sub-criteria.
- `parse_criterion` builds the tree from one line of text with a recursive-descent parser whose precedence is `!` > `&&` > `||`. It raises `ParseError` on bad syntax.

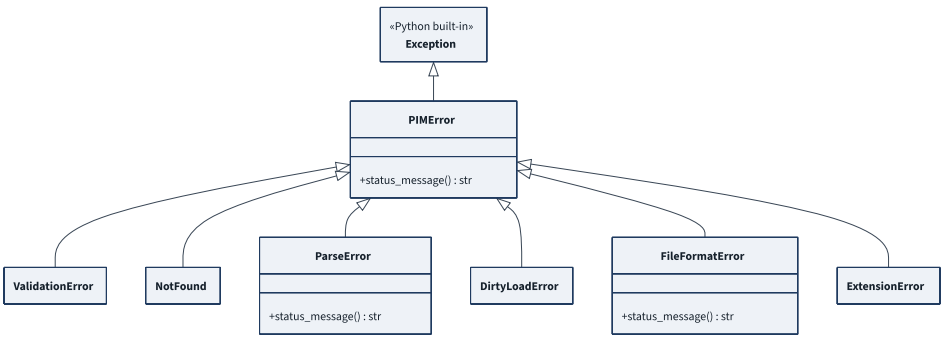


Figure 4. Model classes, part 3: the exception hierarchy.

Every model failure is a subclass of `PIMError`, whose `status_message()` gives the English text that the Controller shows. `ParseError` and `FileFormatError` override it to add a prefix to the message.

3.1.1 Class `PIM` (module `model.pim`)

The Working Collection: PIR identity, search, persistence, and due alarms.

Fields (private):

- `_pirs: dict[int, PIR]`: the PIRs, keyed by Id.
- `_next_id: int`: the next Id to assign.
- `_dirty: bool`: whether there are unsaved changes.
- `_bound_path: str | None`: the Bound File.

Return type	Method	Arguments	Exceptions	Description
`Note`	`create_note`	`text: str | None`	`ValidationError`	Creates and inserts a Note with the next Id, and marks the collection dirty.
`Task`	`create_task`	`description: str | None, deadline: datetime | str | None = None`	`ValidationError`	Creates a Task; the deadline is optional.
`Event`	`create_event`	`description: str | None, start: datetime | str | None, alarms: list[AlarmSpec] | None = None`	`ValidationError`	Creates an Event and rejects alarm times out of range (FR-8).
`Contact`	`create_contact`	`name: str | None, address: str | None = None, mobile: str | None = None`	`ValidationError`	Creates a Contact.
`PIR`	`modify`	`pir_id: int | str, fields: dict[str, object]`	`NotFound, ValidationError`	Copies the PIR and applies the fields to the copy. The copy replaces the original only if it is valid and different; otherwise the original is returned unchanged.
`None`	`delete`	`pir_id: int | str`	`NotFound`	Removes the PIR. Its Id is never reused.
`PIR`	`get`	`pir_id: int | str`	`NotFound`	Returns the PIR with this Id.
`list[PIR]`	`all`	—	—	Every PIR, in Id order.
`list[PIR]`	`search`	`criterion: Criterion`	—	The PIRs for which `criterion.matches(pir)` is true, in Id order.
`list[DueAlarm]`	`due_alarms`	`now: datetime | str`	`ValidationError`	The OVERDUE and SOON alarms at the given `now`. Never reads the clock.

Return type	Method	Arguments	Exceptions	Description
`None`	`save`	`path: str | os.PathLike[str]`	`ValidationError`, `OSError`	Writes the PIM File atomically, adding `.pim` if it is missing, expanding a leading ~, and creating missing folders. Binds the path and clears the dirty flag.
`None`	`load`	`path: str | os.PathLike[str]`, `force: bool = False` (keyword)	`ExtensionError`, `ValidationError` `DirtyLoadError`, `FileFormatError`	Replaces the collection from a file. The file is parsed completely first, so a bad file changes nothing.
`bool`	`is_dirty`	—	—	True if there are unsaved changes.
`str | None`	`bound_path`	—	—	The path of the last successful save or load.

3.1.2 Class `PIR` (abstract) and subclasses (module `model.pir`)

The base class of all records.

Fields:

- `id: int`
- `type_name: str` (class constant)
- `FIELDS: tuple[FieldSpec, ...]` (class constant)

Return type	Method	Arguments	Exceptions	Description
`str`	`display_name` (property)	—	—	The derived list label (SRS glossary).
`list[str]`	`text_fields`	—	—	Every text value that is set, used by unqualified `contains`.
`str | None`	`text_value`	`field: str`	—	The named text field, or `None` if this type has no such field or it is not set.
`list[datetime]`	`time_values`	`field: str`	—	The instants of `deadline`, `start`, or `alarm` (Effective Alarm Times); empty if there are none.
`datetime | None`	`relevant_time`	—	—	The deadline or start shown in the list; `None` for a Note or Contact.
`str`	`display_field`	`key: str`	—	The current value of a field as a prompt hint: `none` if unset, `n alarm(s)` for the alarm list.
`None`	`modify`	`fields: dict[str, object]`	`ValidationError`	Applies field changes in place, validating them all before assigning any. Rejects a change of type.

Return type	Method	Arguments	Exceptions	Description
`dict`	`to_json`	—	—	The `pim/v1` JSON object.
`list[tuple[str, str]]`	`detail_lines`	—	—	The (label, value) rows printed by `print`.

The subclass constructors:

Subclass	Constructor arguments	Extra fields	Exceptions
`Note`	`pir_id: int, text: str | None`	`text: str`	`ValidationError`
`Task`	`pir_id: int, description: str | None, deadline: datetime | str | None = None`	`description: str, deadline: datetime | None`	`ValidationError`
`Event`	`pir_id: int, description: str | None, start: datetime | str | None, alarms: list[AlarmSpec] | None = None`	`description: str, start: datetime, alarms: list[RelativeAlarm | AbsoluteAlarm]`	`ValidationError`
`Contact`	`pir_id: int, name: str | None, address: str | None = None, mobile: str | None = None`	`name: str, address: str | None, mobile: str | None`	`ValidationError`

`Event` adds one method, `effective_alarm_times() -> list[datetime]`.

3.1.3 Alarm and field classes (module `model.pir`)

Class	Constructor	Fields	Methods
`RelativeAlarm`	`amount: int | str, unit: str`. Raises `ValidationError` for a negative or non-integer amount, or for a unit other than minute, hour, day, or week (singular or plural, in any case).	`amount: int, unit: str`	`effective(start: datetime) -> datetime` (start minus the duration); `to_json() -> dict`; `kind_label() -> str`
`AbsoluteAlarm`	`at: datetime | str`. Raises `ValidationError`.	`at: datetime`	`effective(start: datetime) -> datetime` (returns `at`); `to_json() -> dict`; `kind_label() -> str`
`DueAlarm`	`event_id: int, alarm_index: int, at: datetime, status: str, description: str`	the same	`key() -> tuple[int, int]` (the identity used by `dismiss`)
`FieldSpec` (dataclass)	`key: str, label: str, kind: str, required: bool = True`	the same	—

`AlarmSpec` stands for `RelativeAlarm | AbsoluteAlarm | dict`, where a `dict` is the JSON form read from a file.

3.1.4 Criterion classes (module `model.criterion`)

Class	Constructor	Fields	`matches(pir: PIR) -> bool` returns true when …
`Criterion` (abstract)	—	—	(raises `NotImplementedError`)
`TypeIs`	`type_name: str`. Raises `ParseError`.	`type_name: str`	the PIR's type is `type_name`
`Contains`	`field: str | None`, `needle: str`. Raises `ParseError`.	`field: str | None`, `needle: str` (case-folded)	the folded field value, or any text field when `field` is `None`, contains `needle`
`TimeCompare`	`field: str`, `op: str`, `instant: datetime`. Raises `ParseError`.	`field`, `op`, `instant`	any instant of `field` satisfies `op` against `instant`; false if there is none
`And` / `Or`	`left: Criterion`, `right: Criterion`	`left`, `right`	both children match / either child matches; the right child is evaluated only when needed
`Not`	`inner: Criterion`	`inner`	the child does not match

3.1.5 Module-level functions and constants of `model`

Return type	Function	Module	Arguments	Exceptions	Description
`Criterion`	`parse_criterion`	`criterion`	`text: str`	`ParseError`	Parses one criterion line (grammar in SRS FR-16).
`list[tuple str]]`	`tokenize`	`criterion`	`text: str`	`ParseError`	Splits a criterion line into (kind, value) tokens.
`datetime`	`parse_datetime`	`pir`	`value: datetime | str, field: str = "datetime"`	`ValidationError`	Parses ISO 8601 or `YYYY-MM-DD HH:MM`, defaults to HKT, and drops seconds. A blank value fails with `<field> is required`.
`datetime | None`	`parse_optional_datetime`	`pir`	`value: datetime | str | None`	`ValidationErr`	As `parse_datetime`, but a blank value or `none` gives `None`.
`str`	`format_datetime`	`pir`	`dt: datetime`	`ValidationError`	ISO 8601 text with the offset, to the minute (seconds are always `:00`). Rejects a datetime without a time zone.
`type[PIR] | None`	`pir_class`	`pir`	`type_name: str`	—	The PIR subclass for a type name, or `None`.
`PIR`	`pir_from_json`	`pir`	`data: dict[str, object]`	`FileFormatError`, `ValidationError`	Rebuilds a PIR from its `pim/1` object.
`Path`	`require_pim_extension`	`pimfile`	`path: str | os.PathLike[str]`	`ExtensionErro`	The path, if it ends in `.pim`.

Return type	Function	Module	Arguments	Exceptions	Description
`Path`	`append_pim_extension`	`pimfile`	`path: str | os.PathLike[str]`	`ValidationError`	The path with `.pim` added if missing and ~ expanded; rejects an empty file name.
`str`	`dump`	`pimfile`	`next_id: int, pirs: list[PIR]`	—	The text of a PIM File.
`tuple[int, list[PIR]]`	`read_pim_file`	`pimfile`	`path: str | os.PathLike[str]`	`ExtensionError`, `FileFormatError`	Parses a PIM File without touching any collection.
`Path`	`write_pim_file`	`pimfile`	`path: str | os.PathLike[str], next_id: int, pirs: list[PIR]`	`ValidationErr` `OSError`	Creates missing folders, writes a temporary file, and atomically replaces the target.

Constants: `TYPE_NAMES` (the four type names), `HKT` (the default time zone, or a fixed UTC+8 zone without a time-zone database), and the field kinds `KIND_TEXT`, `KIND_DATETIME`, and `KIND_ALARMS` used in `FieldSpec`.

3.1.6 Exceptions

All exceptions are subclasses of `PIMError(Exception)`, which provides `status_message() -> str`:

- `ValidationError`: a missing or illegal field value.
- `NotFound`: no PIR has this Id.
- `ParseError`: bad criterion syntax; its status message starts with `search syntax error:`.
- `DirtyLoadError`: a load with unsaved changes and without `force`.
- `FileFormatError`: the file is not a valid `pim/v1` PIM File; its status message starts with `not a PIM file:`.
- `ExtensionError`: the path does not end in `.pim`.

3.1.7 PIM File format

A PIM File is UTF-8 JSON. `write_pim_file` writes it, and `read_pim_file` checks it completely before `PIM.load` replaces anything (SRS FR-31, FR-36).

```
{
  "format": "pim/v1",
  "next_id": 5,
  "pirs": [
    { "id": 1, "type": "note", "text": "buy milk" },
    { "id": 2, "type": "task", "description": "submit ZIP",
      "deadline": "2026-11-20T20:00:00+08:00" },
    { "id": 3, "type": "event", "description": "COMP3211 lecture",
      "start": "2026-09-14T18:30:00+08:00",
      "alarms": [
        { "kind": "relative", "amount": 1, "unit": "day" },
        { "kind": "relative", "amount": 0, "unit": "minute" },
        { "kind": "absolute", "at": "2026-09-13T09:00:00+08:00" } ] },
    { "id": 4, "type": "contact", "name": "Ada",
      "address": null, "mobile": "12345678" }
  ]
}
```

- `unit` is `minute`, `hour`, `day`, or `week`; an `amount` of 0 means at the start.
- The writer always writes every field, with `null` for an unset optional field and `[]` for an Event without alarms. The reader also accepts a missing optional field.
- `next_id` is stored so that a deleted Id is never reused after a load. If it is not greater than the largest Id in the file, the load raises it.

3.2 Controller and View classes

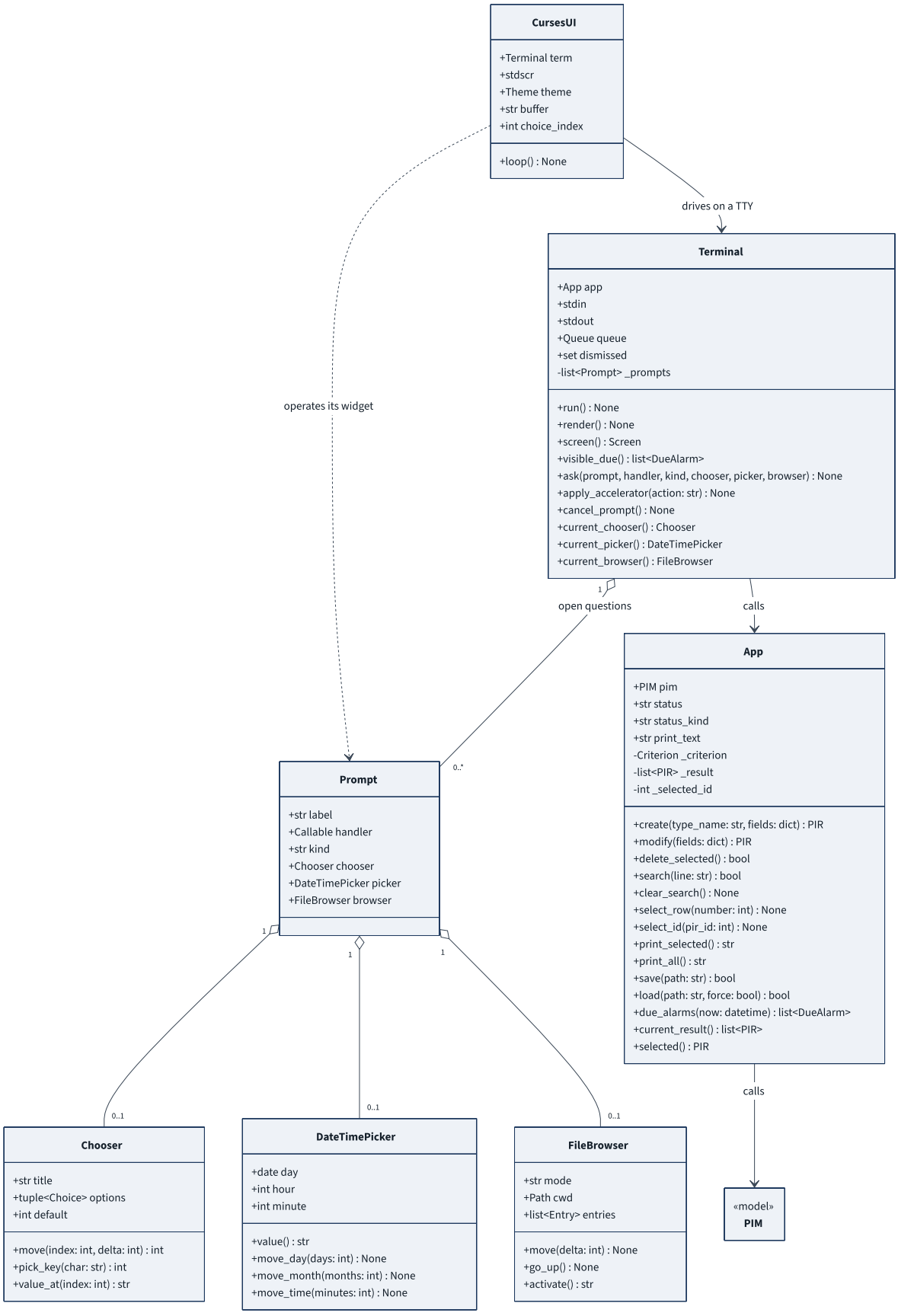


Figure 5. Controller and View classes. Argument types are shortened; the tables give the full types.

Relationships

- `Terminal` calls `App` for every user action, and `App` calls `PIM`.
- `CursesUI` drives a `Terminal` when a TTY is available. The two classes form one front end: `CursesUI` handles keys and drawing, and passes each finished answer to the `Terminal`'s internal handlers (`_handle_line`, `_handle_eof`, `_handle_interrupt`).
- `Terminal` keeps a stack of open `Prompt` records. A prompt may hold one widget: a `Chooser`, a `DateTimePicker`, or a `FileBrowser`. `CursesUI` operates the widget; the widget only produces the answer text, which the prompt's handler checks exactly like typed input.

Neither `App` nor `Terminal` stores PIR data of its own. For the Current Result, `App` keeps a list of references to PIRs held by `PIM`, and it runs the active criterion again after every change.

3.2.1 Class `App` (module `controller.app`)

Fields:

- `pim: PIM`
- `status: str` and `status_kind: str` (`info`, `ok`, or `err`)
- `print_text: str`: the text of the last `print` or `print all`
- private: `_criterion: Criterion | None`, `_criterion_line: str | None`, `_result: list[PIR]`, `_selected_id: int | None`

Constructor: `App(pim: PIM)`. It binds to a Working Collection, which may be empty or already loaded, and shows the whole collection as the Current Result.

The controller reports failures instead of raising them: it sets `status` to one English line with `status_kind err` and returns `None` or `False`. The two exceptions are `save_target`, which the View calls to check a path before asking about an overwrite, and `due_alarms`, which passes on the model's `ValidationError` for an invalid `now`.

Return type	Method	Arguments	Exceptions	Description
`PIR | None`	`create`	`type_name: str, fields: dict[str, object]`	—	Creates a PIR and selects it.
`PIR | None`	`modify`	`fields: dict[str, object]`	—	Modifies the Selection. Empty `fields` change nothing.
`bool`	`delete_selected`	—	—	Deletes the Selection after the View has confirmed.
`bool`	`search`	`line: str`	—	Parses and runs a criterion. On an error, keeps the Current Result.
`None`	`clear_search`	—	—	Restores the whole collection as the Current Result.
`None`	`select_row`	`number: int | str`	—	Selects by row of the Current Result.
`None`	`select_id`	`pir_id: int | str`	—	Selects by Id.
`str | None`	`print_selected`	—	—	The detail text of the Selection.

Return type	Method	Arguments	Exceptions	Description
`str`	`print_all`	—	—	The detail text of every PIR in the Current Result.
`None`	`clear_print`	—	—	Drops the last printed text.
`bool`	`save`	`path: str | os.PathLike[str] | None = None`	—	Saves to the Bound File or to `path`; the status names the absolute path. Reports an `OSError` as a status.
`bool`	`load`	`path: str | os.PathLike[str], force: bool = False`	—	Loads, then resets the search and the Selection; the status names the absolute path.
`str`	`save_target`	`path: str | os.PathLike[str]`	`ValidationError`	The path that `save` would write, with `.pim` added.
`bool`	`would_overwrite`	`path: str | os.PathLike[str]`	—	Whether a save would replace an existing file other than the Bound File.
`list[DueAlarm]`	`due_alarms`	`now: datetime`	`ValidationError`	Passes the call on to the model.
`None`	`set_status`	`text: str, kind: str = "info"`	—	Sets the status line and its tone.
`list[PIR]`	`current_result`	—	—	The Current Result, for drawing.
`PIR | None`	`selected`	—	—	The selected PIR.
`int | None`	`selected_id`	—	—	The Id of the selected PIR.
`bool`	`has_criterion`	—	—	Whether a search is active.
`str | None`	`criterion_line`	—	—	The text of the active search.
`str | None`	`bound_path`	—	—	The Bound File.
`bool`	`is_dirty`	—	—	Whether there are unsaved changes.

Module `controller.app` also defines `format_pir(pir: PIR) -> str`, which renders one PIR as `key: value` lines for `print`, the command summary `HELP`, and the status tones `STATUS_INFO`, `STATUS_OK`, and `STATUS_ERR`.

`controller.errors.message_for(exc: BaseException) -> str` maps an exception to one status line:

- a `PIMError` → its `status_message()`;
- an `OSError` → its text;
- anything else → `command failed`.

3.2.2 Class `Terminal` (module `view.terminal`)

Fields:

- `app: App`

- `stdin: TextIO`, `stdout: TextIO`
- `queue: Queue`: lines from the standard-input reader thread (line interface)
- `dismissed: set[tuple[int, int]]`: dismissed alerts, keyed by (Event Id, alarm index)
- `page_size: int`
- private: `_now: datetime | Callable[[], datetime] | None`, `_prompts: list[Prompt]` (a stack: the newest prompt is answered first), `_running: bool`, `_last_snapshot`

Constructor: `Terminal(app: App, stdin: TextIO | None = None, stdout: TextIO | None = None, now: datetime | Callable[[], datetime] | None = None)`. Tests inject the streams and `now`; when they are left out, the process streams and the HKT wall clock are used.

These methods do not catch exceptions themselves. Every command runs inside a handler of the line loop, or inside `CursesUI._safe`, that turns any exception into the status line (NFR-4).

Return type	Method	Arguments	Description
`None`	`run`	—	Starts `CursesUI` on a TTY, or otherwise the line loop. Returns on quit.
`list[DueAlarm`	`visible_due`	—	The due alarms minus the dismissed ones.
`Screen`	`screen`	—	The screen model (title, alerts, list, details, status) that both front ends draw.
`None`	`render`	—	Draws the line-interface screen.
`None`	`ask`	`prompt: str`, `handler: Callable[[str], object]`, `kind: str | None = None`, `chooser: Chooser | None = None`, `picker: DateTimePicker | None = None`, `browser: FileBrowser | None = None`	Opens a prompt; `handler(answer)` runs when the user answers. By convention at most one widget is given.
`None`	`apply_accelerator`	`action: str`	Runs the command bound to a single key.
`None`	`cancel_prompt`	—	Esc: drops the current wizard, but not an unsaved-changes question.
`None`	`idle_escape`	—	Esc with no wizard open: ends an active search.
`None`	`retry_search_prompt`	—	Opens the criterion field again after a syntax error (FR-23).
`Chooser | None`	`current_chooser`	—	The selector of the current prompt, if any.
`DateTimePicker | None`	`current_picker`	—	The calendar of the current date-time prompt, if any.
`FileBrowser | None`	`current_browser`	—	The folder browser of the current load or save-as prompt, if any.
`None`	`type_path_instead`	—	Closes the browser so that the same prompt takes a typed path.

3.2.3 Class `CursesUI` (module `view.curses_ui`)

Constructor: `CursesUI(terminal: Terminal, stdscr)`, where `stdscr` is the curses window. The module function `run_curses(terminal) -> None` creates it inside `curses.wrapper` and runs `loop()`; `Terminal.run` calls it on a TTY.

Fields: `term: Terminal`, `stdscr`, `theme: Theme`, the edit state `buffer: str` and `cursor: int`, the selector highlight `choice_index: int`, and the overlay state `raw_command`, `show_help`, `print_open`, and `print_scroll`.

Its one public method is `loop() -> None`, the full-screen event loop: `get_wch` with a 500 ms timeout, and a 25 ms Esc delay so that a lone Esc acts at once. Every command runs through `_safe`, which turns any exception into the status line, so `loop` raises nothing to its caller. The module function `title_bar_text(title: str, width: int) -> str` fits the title bar beside the clock, trimming a long file path from the left so that the file name stays visible.

3.2.4 Classes `Chooser` and `Choice` (module `view.widgets`)

A `Chooser` is a closed list of answers, such as Note · Task · Event · Contact, or Yes · No. It is an immutable dataclass; the View keeps the highlighted index, so the methods take and return indexes. The value it submits is the same text a user would type in the line interface.

Fields: `title: str`, `options: tuple[Choice, ...]`, `default: int = 0`, `hint: str = "← →  Enter Esc"`. Each `Choice` has `value: str`, `label: str`, `key: str | None = None` (a one-key shortcut), and `tone: str | None = None` (a colour role).

Return type	Method	Arguments	Exceptions	Description
`int`	`clamp`	`index: int`	—	Keeps `index` inside the option list.
`int`	`move`	`index: int, delta: int`	—	Moves by `delta` without wrapping.
`int | None`	`pick_key`	`char: str`	—	The option whose key is `char`; otherwise, when no option has a digit key, the option at that 1-based digit; `None` if nothing matches.
`str`	`value_at`	`index: int`	—	The value to submit for that option.

3.2.5 Class `DateTimePicker` (module `view.widgets`)

A month grid and a time list for date-time prompts on a TTY (FR-11). The week starts on Monday. In the time list, the time moves in 15-minute steps with ↑ and ↓, in 1-hour steps with ← and →, and in 1-minute steps with `+` and `-`. The submitted value is text in the form `YYYY-MM-DD HH:MM`, which the model reads as HKT.

Fields: `title: str`, `required: bool`, `today: date`, `day: date`, `hour: int`, `minute: int`, `view: date` (the first day of the month shown), `focus: str` (`date` or `time`).

Constructor: `DateTimePicker(title: str, now: datetime, *, initial: datetime | None = None, required: bool = True)`. It starts on `initial`, or otherwise on the first 15-minute mark at or after `now`.

Return type	Method	Arguments	Exceptions	Description
`str`	`value`	—	—	The text to submit, `YYYY-MM-DD HH:MM`.
`str`	`summary`	—	—	A readable confirmation, for example `Mo 14 September 2026 18:30 HKT`.
`str`	`month_title`	—	—	The month name and year for the grid header.
`list[list[date]]`	`weeks`	—	—	The weeks of the month shown, Monday first.
`None`	`move_day`	`days: int`	—	Moves the chosen day and keeps its month in view.
`None`	`move_month`	`months: int`	—	Shows another month, clamping the day if the month is shorter.
`None`	`move_time`	`minutes: int`	—	Changes the time, wrapping within the day.
`None`	`jump_today`	`now: datetime`	—	Chooses today and keeps the time.
`None`	`toggle_focus`	—	—	Switches between the grid and the time list.
`list[tuple[int, int]]`	`time_slots`	`count: int = 7`	—	`count` times around the chosen one, 15 minutes apart.

3.2.6 Class `FileBrowser` (module `view.file_browser`)

The folder browser for load and save-as paths on a TTY (FR-39). It lists one folder: the parent link, the subfolders, and the `.pim` files, leaving out hidden entries. In save mode it adds a "new file in this folder" row. It returns a path string, so the prompt handler checks it like a typed path (Section 5.4).

Fields: `mode: str` (`load` or `save`), `cwd: Path`, `entries: list[Entry]`, `index: int` (the highlighted row), `error: str` (set when the folder cannot be read). Each `Entry` has `kind: str` (`parent`, `folder`, `file`, or `new`) and `name: str`, and its method `label() -> str` gives the row text, with a `/` after a folder name.

Constructor: `FileBrowser(mode: str, start: str | os.PathLike[str])`. It reads `start` at once.

Return type	Method	Arguments	Exceptions	Description
`str`	`title` (property)	—	—	The heading of the overlay.
`None`	`refresh`	—	—	Reads `cwd` again. A folder that cannot be read sets `error` instead of raising.
`Entry | None`	`current`	—	—	The highlighted row.
`None`	`move`	`delta: int`	—	Moves the highlight without wrapping.
`None`	`jump`	`last: bool`	—	Highlights the first or the last row.

Return type	Method	Arguments	Exceptions	Description
`None`	`go_up`	—	—	Opens the parent folder and highlights the folder just left.
`None`	`go_home`	—	—	Opens the home folder.
`str | None`	`activate`	—	—	Opens a folder (and returns `None`), or returns the absolute path of a `.pim` file.
`str`	`highlighted_path`	—	—	The absolute path of the highlighted folder or file.
`bool`	`wants_typing`	—	—	Whether the “new file” row is highlighted.
`str`	`typed_start`	—	—	The text to pre-fill when switching to a typed path: the current folder.
`tuple`	`state`	—	—	A hashable snapshot, so that the screen is redrawn only when it changes.

3.2.7 View helper modules

Module	Role
`view.layout`	Builds the `Screen` model (title, alerts, list rows, details, menu) with `build_screen`, and formats list rows and pane titles.
`view.widgets`	`Chooser`, `Choice`, and `DateTimePicker` (above); `Prompt`, one open question with its handler and at most one widget; the selector factories `type_chooser`, `yes_no_chooser`, `alarm_kind_chooser`, `alarm_amount_chooser`, `alarm_unit_chooser`, and `dirty_chooser`; and `wrap_chips` and `composer_height` for laying out the composer.
`view.file_browser`	`FileBrowser` (above) and its row type `Entry`.
`view.keys`	Maps keys to actions (accelerators), for example `w` save, `W` save as, `o` load.
`view.theme`	Colour roles for 256-colour, 8-colour, and monochrome terminals.
`view.textwidth`	Clipping and padding that respect East Asian character widths.
`view.stdin_reader`	The daemon thread that feeds input lines into the `Queue` in the line interface.

4 Example use: search for records, then update one

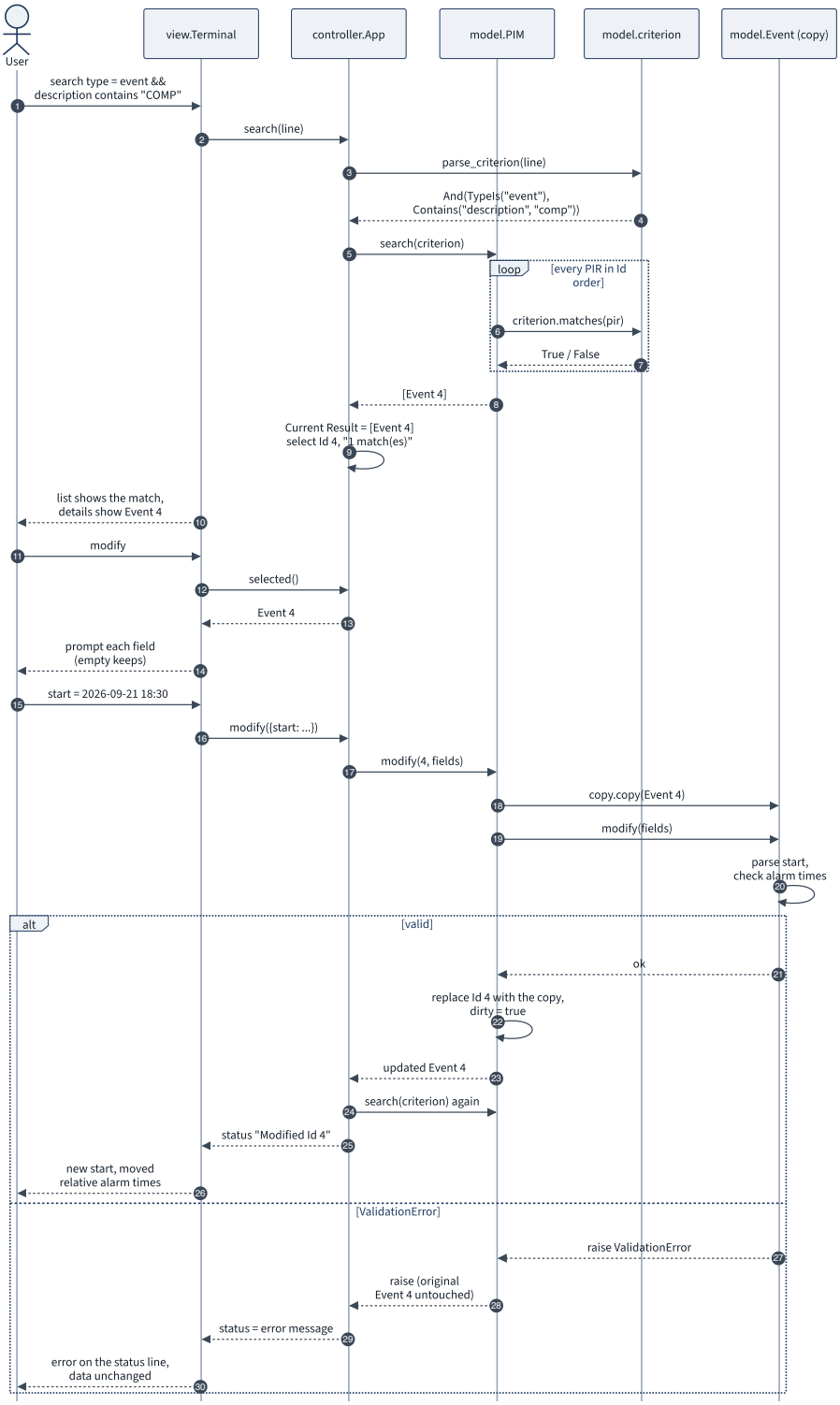


Figure 6. Sequence diagram: searching for Events whose description contains “COMP”, then changing the start of the match.

The numbers in brackets refer to the message numbers in the diagram.

Search (messages 1–10)

- *(1–2)* The user enters a criterion line, and `Terminal` passes the raw line to `App.search`.
- *(3–4)* `App` calls `parse_criterion`, which tokenizes the line and builds the tree `And(TypeIs("event"), Contains("description", "comp"))`.
- *(5–8)* `PIM.search` calls `matches` on every PIR in Id order. `And` evaluates its right child only when the left child matches.
- *(9)* `App` stores the criterion and the matches as the Current Result, selects the first match, and sets the status `1 match(es)`.
- *(10)* The View redraws the list and the details.

A syntax error would raise `ParseError` at message 3. `App` would then leave the Current Result as it was and show `search syntax error: …`.

Update (messages 11–30)

- *(11–15)* `modify` asks `App` for the Selection. `Terminal` then prompts field by field from the Event's `FIELDS`; an empty answer keeps a value.
- *(16–17)* `Terminal` passes only the answered fields to `App.modify`, which forwards them to `PIM.modify`.
- *(18–20)* The model uses **copy-then-replace**: it copies the Event and applies the change to the copy, which parses the new start and checks that every Effective Alarm Time is still in range.
- *(21–26)* **If the copy is valid:** `PIM` puts the copy in place of the original under the same Id and marks the collection dirty. `App` runs the stored criterion again, so the Current Result is up to date, and the View shows the new start and the moved relative alarm times. (If the copy equals the original, `PIM` returns the original and the collection stays clean.)
- *(27–30)* **If validation fails:** the exception propagates before the original is touched. `App` shows the message, and the data is exactly as before (FR-15, NFR-4).

The diagram shows the main design points of the system:

- the criterion is parsed and evaluated inside the model;
- the Controller holds the Current Result;
- the Id never changes;
- relative alarms follow the start;
- a failed update changes nothing.

5 Design decisions

Each table records one group of decisions: what was decided, why, and what was rejected. Section 2.1 covers the choice of MVC itself.

5.1 Platform and libraries

Decision	Why	Rejected
Python 3.12 with the standard library only.	The brief allows Java or Python and requires the standard library only. Python is the language the group knows best.	Java, which the brief does not favour. Any pip package, for example Textual, Rich, or pytest.
A full-screen `curses` interface on an interactive terminal, and a plain line interface when input or output is redirected (or `PIM_NO_CURSES=1` is set).	The brief asks for a command-line system and gives no credit for a GUI. `curses` is part of the standard library. The line interface lets tests inject `stdin`, `stdout`, and `now`.	A GUI window (tkinter). A third-party terminal library. A line interface only, which would make alarms and the Selection hard to see.
Every time is time-zone aware. A time typed without an offset is Hong Kong Time (`Asia/Hong_Kong`), and comparisons use instants. Without a time-zone database on the computer, HKT is a fixed UTC+8 offset.	The same PIM File then means the same instants on every machine. Hong Kong has had no daylight saving since 1979, so the fixed offset gives the same present-day instants, and the program still starts on a computer without the database.	The machine's local zone. Requiring an offset on every input. Refusing to start without the database. The third-party `tzdata` package.

5.2 Structure

Decision	Why	Rejected
Three top-level packages, `model`, `view`, and `controller`, plus `pim.py` as the composition root (Section 2.2).	The brief requires a package named exactly `model`, and the other components must be easy to identify.	A nested `pim.model`, which is not the required name. View and controller code inside `pim.py`.
`model` keeps every rule behind a small interface: `PIM`, `parse_criterion`, and value classes (Section 2.2).	Validation, matching, and the JSON codec stay inside, so each rule sits in one place and is unit-tested without the screen.	Loose procedural functions. A service class over plain records. A `Storage` interface with a single implementation.
The View owns an in-process event loop with a 500 ms tick. The model never reads the clock: `now` is passed in (Section 2.2).	Alarm Alerts appear while the user is idle, without a second process, and alarm tests control time exactly.	Blocking on `input()`. A background process or operating-system notifications.

5.3 Data and rules

Decision	Why	Rejected
A PIM File is UTF-8 JSON (`"format": "pim/v1"`). `save` adds `.pim` if it is missing, and `load` rejects any other extension.	The brief fixes only the `.pim` extension. JSON is in the standard library, readable, and easy to test (SRS FR-31).	`pickle`, which is opaque and unsafe to load. YAML, which needs a third-party parser. A custom text format, which needs its own parser.

Decision	Why	Rejected
A system-assigned integer Id is the only identity. It is stored in the file and never reused.	Row numbers change with every search, and descriptions and Contact names repeat (for example a weekly class).	A unique label chosen by the user. Selecting by row position alone.
The type of a PIR cannot change.	US6 modifies the data of a PIR, and the fields of different types do not map onto each other.	Converting a PIR in place. Instead, the user deletes it and creates one of the new type.
An Event has zero or more alarms. Each is *Relative* (at the start, or a whole number of units before it) or *Absolute* (a fixed instant), as in the iCalendar `TRIGGER` property.	Several reminders per event is the standard model. Relative alarms move with the start; absolute ones do not.	Exactly one alarm. Relative alarms after the start, which an absolute alarm already covers. The `.ics` file format.
No recurring events.	Appendix B does not ask for them. A series needs a second life cycle (one occurrence or the whole series) and earns no credit.	Repeat rules (RRULE). A weekly class is entered as several Events.
`contains` is a substring test after `str.casefold()` on both sides.	US7 asks for a substring test. Ignoring case is kinder to the user, and case folding does not depend on the locale.	Fuzzy matching. Locale-dependent comparison.
A failed command changes nothing and shows one English status line.	Invalid input is normal at a command line. The model validates a copy (`modify`) or parses the whole file (`load`) before it replaces anything, so each error requirement can be tested (SRS NFR-4).	Partial updates. Printing a traceback or ending the program.

5.4 Interaction

Decision	Why	Rejected
Create and modify ask for one field at a time. Search takes one criterion line (grammar in SRS FR-16).	Prompts need no syntax to remember. A criterion must be exact to be testable, especially `&&`, `||`, `!`, and time comparisons.	All fields on one command line. A menu-driven query builder.
Commands are verbs (`create`, `search`, `modify`, `print`, `delete`, `save`, `load`, …). On a terminal, single keys such as `c`, `/`, `w`, `W`, and `o` are shortcuts for the same verbs.	The verbs map one-to-one onto Appendix B, so the SRS, the user manual, and the screen use one list of commands.	Numbered menus. A separate key-only command language.
On a terminal, the load and save-as prompts open a folder browser that lists folders and `.pim` files. `/` or Tab switches to a typed path.	Users expect to pick a location, and a bare `path:` field gave no hint of how to open a file. The chosen path goes to the same handler as a typed one, so the `.pim` rule, the overwrite question, and the handling of unsaved changes stay in one place.	A Finder dialog through `osascript`, or tkinter's file dialog: both are GUI windows.

5.5 Testing

Decision	Why	Rejected
`unittest` in three layers: unit, integration, and end-to-end. The model unit tests execute every countable line of `model/`. A `hooks/pre-commit` script refuses a commit unless all three layers pass.	The brief grades the model's unit tests. Integration tests and scripted sessions through `Terminal.run()` catch breaks between packages. `coverage_report.py` measures lines with the standard `trace` module, so everything stays in the standard library.	pytest and `coverage.py`, which are third-party. Testing the model only.

6 Design quality

Modularity

- Each package has one reason to change: the screen (`view`), the command flow (`controller`), or the rules and data (`model`).
- `model` exposes a small interface (Section 2.2) and keeps the JSON codec, the tokenizer, and the matching logic behind it.
- Unit tests that never touch the View execute every countable line of `model/`.

Efficiency

- Search, the alarm check, save, and load each take time linear in the number of PIRs.
- With 10,000 PIRs they take 2 ms, 2 ms, 44 ms, and 24 ms on the development Mac (SRS NFR-7).
- The View redraws only when its screen snapshot changes, so the 500 ms tick causes no flicker.

Extendibility

- *A new criterion* is a new `Criterion` subclass plus one branch in the parser; a new searchable field is one entry in the parser's list of text or time fields.
- *A new PIR type* is a `PIR` subclass with its `FIELDS`, plus its registrations in the model (the name in `TYPE_NAMES` and `pir_class`, one branch in `pir_from_json`, and one `PIM.create_*` method) and one entry in the controller's `_CREATE` table. The View generates the field prompts from `FIELDS`; only its type selector, type pin and colour, and the help text need the new name.
- *A new front end* can reuse `App` unchanged.

Justifiability. Section 2.1 explains why the system uses MVC, and Section 5 gives the reason and the rejected alternatives for every other decision that is hard to reverse.